Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

int main() {
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("\n");

    int at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n];
    int completed[n];

    // Initialize
    for(i = 0; i < n; i++) {
        completed[i] = 0;
    }

    // Input
    for(i = 0; i < n; i++) {
        printf("Enter AT, BT and Priority for P%d: ", i + 1);
        scanf("%d %d %d", &at[i], &bt[i], &pr[i]);
        printf("\n");
    }

    int time = 0, done = 0;
    float total_tat = 0, total_wt = 0;
```

Test Cases 2

Run Sample Cases

Run All Cases

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Select Language : C (GCC 9.2.0)

```
    float total_tat = 0, total_wt = 0;

    while(done < n) {
        int idx = -1;
        int best_pr = 100000;

        // Find highest priority (lowest value)
        for(i = 0; i < n; i++) {
            if(at[i] <= time && completed[i] == 0) {
                if(pr[i] < best_pr) {
                    best_pr = pr[i];
                    idx = i;
                }
                // Tie-breaker: earlier arrival
                else if(pr[i] == best_pr) {
                    if(at[i] < at[idx]) {
                        idx = i;
                    }
                }
            }
        }

        if(idx == -1) {
            time++; // CPU idle
        } else {
            time += bt[idx];
            ct[idx] = time;
```

Test Cases 2

Run Sample Cases

Run All Cases

Completed

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

- The first integer denotes the Arrival Time (AT)
- The second integer denotes the Burst Time (BT)
- The third integer denotes the Priority of the process.

A lower numerical value indicates a higher priority.

Output Format

The output displays the Average Turnaround Time and the Average Waiting Time.

Select Language : C (GCC 9.2.0)

```
                    idx = i;
                }
            }
        }
    }

    if(idx == -1) {
        time++; // CPU idle
    } else {
        time += bt[idx];
        ct[idx] = time;
        completed[idx] = 1;
        done++;

        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];

        total_tat += tat[idx];
        total_wt += wt[idx];
    }
  }

  printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
  printf("Average Waiting Time = %.2f\n", total_wt / n);

  return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases